

	Guía Unificada de Laboratorios	Código	FLA-23 v.00
		Página	1 de 1

1. Título

DESARROLLO DE UNA APLICACIÓN WEB DE BÚSQUEDA DE POKÉMON UTILIZANDO ANGULAR Y POKEAPI

2. Objetivo

Desarrollar una aplicación web interactiva usando Angular que permita buscar y visualizar información detallada de Pokémon mediante la integración con la API pública PokeAPI.

3. Marco Teórico

Las **Interfaces de Programación de Aplicaciones (APIs)** son mecanismos que permiten la comunicación entre diferentes sistemas mediante solicitudes HTTP estructuradas en formato JSON. En el contexto del desarrollo web moderno, **Angular** provee el módulo HttpClient para facilitar la integración con APIs RESTful, posibilitando operaciones asíncronas como consultas GET y POST con tipado estricto de datos. Por otra parte, **PokéAPI** es un servicio REST gratuito que proporciona datos estructurados sobre el universo Pokémon, incluyendo atributos como nombre, tipo, estadísticas y representaciones visuales. Su simplicidad y estructura JSON la hacen ideal para experimentación académica en consumo de servicios RESTful desde frameworks frontend modernos. En este proyecto, se explora el uso del módulo HttpClientModule para obtener y visualizar información dinámica proveniente de la API, utilizando el **patrón de arquitectura por componentes y servicios** característico de Angular. Además, se emplea **Postman** como herramienta auxiliar para verificar la comunicación y respuesta de la API antes de integrarla en el código, asegurando una conexión confiable y la comprensión del esquema de datos retornado.

3.1. Conceptos Fundamentales

Término	Definición	Analogía
Component	Bloque de construcción básico que controla una parte de la pantalla (vista)	Como un widget en una aplicación móvil
Module	Contenedor que agrupa componentes, servicios y otras funcionalidades relacionadas	Como una carpeta que organiza archivos relacionados
Service	Clase que contiene lógica de negocio y datos compartidos	Como una biblioteca que ofrece funciones específicas
Directive	Clase que modifica la apariencia o comportamiento de elementos del DOM	Como instrucciones para modificar elementos HTML
Pipe	Transformador de datos en las plantillas	Como un filtro de Excel para formatear datos
Decorator	Anotación que añade metadata a clases, propiedades o métodos	Como etiquetas que describen el propósito de algo
Template	Vista HTML del componente	El diseño visual de una página
Data Binding	Sincronización entre el modelo de datos y la vista	Conexión bidireccional entre datos y pantalla
Dependency Injection	Patrón para proveer dependencias a las clases	Como solicitar herramientas en lugar de crearlas
Observable	Objeto que emite datos a lo largo del tiempo	Como una suscripción a notificaciones

3.2. Decoradores Principales

Decorador	Uso	Ejemplo
@Component	Define un componente	@Component({ selector: 'app-root' })
@NgModule	Define un módulo	@NgModule({ declarations: [...] })
@Injectable	Define un servicio inyectable	@Injectable({ providedIn: 'root' })
@Input	Recibe datos del componente padre	@Input() nombre: string
@Output	Envía eventos al componente padre	@Output() click = new EventEmitter()
@ViewChild	Accede a elementos de la vista	@ViewChild('input') input
@HostListener	Escucha eventos del elemento host	@HostListener('click')

3.3. Directivas Estructurales

Directiva	Propósito	Sintaxis
*ngIf	Renderizado condicional	<div *ngIf="condicion">
*ngFor	Iteración sobre listas	<li *ngFor="let item of items">
*ngSwitch	Múltiples condiciones	<div [ngSwitch]="valor">

3.4. Directivas de Atributo

Directiva	Propósito	Sintaxis
[ngClass]	Añade clases CSS dinámicas	[ngClass]="{'activo': esActivo}"
[ngStyle]	Añade estilos inline dinámicos	[ngStyle]="{'color': color}"
[(ngModel)]	Two-way data binding	[(ngModel)]="nombre"

4. Materiales, Equipos e Insumos

Materiales

Tipo	Descripción
Hardware	PC con procesador Dual-Core, mínimo 4 GB RAM y 10 GB de espacio libre.
Sistema Operativo	Windows 10 o superior (64 bits recomendado).
Software	Node.js, npm, Angular CLI, Visual Studio Code, Git.
Extensiones VSCode	Angular Language Service, Angular Snippets, ESLint, Prettier, Path Intellisense, Bootstrap 4 Snippets.
Recursos institucionales	Malla curricular oficial 2024, logotipo del programa, paleta de colores y datos de contacto.

5. Procedimiento

Paso 1. Crear el proyecto en Angular

```
ng new pokemon-search
cd pokemon-search
ng serve
```

Explicación:

El comando ng new crea una nueva estructura de proyecto Angular con las carpetas src/app, src/assets, src/environments, entre otras.
ng serve ejecuta el servidor local (por defecto en <http://localhost:4200>).

Paso 2. Instalar dependencias necesarias

```
npm install
```

Explicación:

Descarga las dependencias definidas en package.json. No se requieren librerías externas aparte de Angular.

Paso 3. Crear componentes y servicio

```
ng generate service services/pokemon
```

ng generate component components/search
ng generate component components/pokemon-card

Estructura generada:

src/app/services/pokemon.service.ts
src/app/components/search/search.component.ts
src/app/components/pokemon-card/pokemon-card.component.ts

Explicación:

Los servicios (pokemon.service.ts) encapsulan la lógica de comunicación con la API.
Los componentes (search y pokemon-card) se encargan de la interfaz y visualización de datos.

Paso 4. Configurar HttpClientModule y FormsModule

Ruta: src/app/app.module.ts

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { HttpClientModule } from '@angular/common/http';
import { FormsModule } from '@angular/forms';
import { AppComponent } from './app.component';
import { SearchComponent } from './components/search/search.component';
import { PokemonCardComponent } from './components/pokemon-card/pokemon-card.component';

@NgModule({
  declarations: [AppComponent, SearchComponent, PokemonCardComponent],
  imports: [BrowserModule, HttpClientModule, FormsModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Explicación:

HttpClientModule habilita la inyección del cliente HTTP para consumir APIs.
FormsModule permite la vinculación de datos (ngModel) en formularios.

Paso 5. Implementar el servicio de conexión a PokéAPI

Ruta: src/app/services/pokemon.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Pokemon {
  id: number;
  name: string;
  sprites: { front_default: string };
  types: Array<{ slot: number, type: { name: string } }>;
  stats: Array<{ base_stat: number, stat: { name: string } }>;
}
```

```

}

@Injectable({
  providedIn: 'root'
})
export class PokemonService {
  private baseUrl = 'https://pokeapi.co/api/v2/pokemon';

  constructor(private http: HttpClient) {}

  getPokemon(nameOrId: string | number): Observable<Pokemon> {
    const url = `${this.baseUrl}/${nameOrId}`;
    return this.http.get<Pokemon>(url);
  }
}

```

Explicación:

- `getPokemon()` forma la URL final de la API (`/pokemon/{id|nombre}`) y retorna un Observable con la respuesta.
- El tipado `Pokemon` permite acceder a los campos de la respuesta JSON de forma segura.

Paso 6. Crear el componente de búsqueda

Ruta: `src/app/components/search/search.component.ts`

```

import { Component } from '@angular/core';
import { PokemonService, Pokemon } from '../services/pokemon.service';

@Component({
  selector: 'app-search',
  templateUrl: './search.component.html',
  styleUrls: ['./search.component.css']
})
export class SearchComponent {
  query: string = '';
  pokemon: Pokemon | null = null;
  error: string | null = null;
  loading = false;

  constructor(private pokemonService: PokemonService) {}

  buscar() {
    if (!this.query) { this.error = 'Ingresa un nombre o ID'; return; }
    this.error = null;
    this.loading = true;
    this.pokemonService.getPokemon(this.query.toLowerCase().trim())
      .subscribe({
        next: (p) => { this.pokemon = p; this.loading = false; },
        error: () => {
          this.loading = false;

```

```

        this.pokemon = null;
        this.error = 'Pokémon no encontrado o error de conexión';
    }
    });
}
}

```

Plantilla: src/app/components/search/search.component.html

```

<div class="search-box">
  <input [(ngModel)]="query" placeholder="Ejemplo: pikachu o 25" />
  <button (click)="buscar()">Buscar</button>
</div>

<div *ngIf="loading">Cargando...</div>
<div *ngIf="error" class="error">{{ error }}</div>

<app-pokemon-card *ngIf="pokemon" [pokemon]="pokemon"></app-pokemon-card>

```

Explicación:

- [(ngModel)] enlaza el valor del input a la variable query.
- buscar() llama al servicio y maneja el resultado asíncrono.
- Se muestra el componente pokemon-card si se obtiene un resultado.

Paso 7. Crear el componente de visualización

Ruta: src/app/components/pokemon-card/pokemon-card.component.ts

```

import { Component, Input } from '@angular/core';
import { Pokemon } from '../../services/pokemon.service';

@Component({
  selector: 'app-pokemon-card',
  templateUrl: './pokemon-card.component.html',
  styleUrls: ['./pokemon-card.component.css']
})
export class PokemonCardComponent {
  @Input() pokemon!: Pokemon;

  get types(): string[] {
    return this.pokemon.types.map(t => t.type.name);
  }

  get mainStats() {
    return this.pokemon.stats.slice(0, 3).map(s => ({name: s.stat.name, value: s.base_stat}));
  }
}

```

Plantilla: src/app/components/pokemon-card/pokemon-card.component.html

```

<div class="card">

```

```

<h2>{{ pokemon.name | titlecase }} (#{ {{ pokemon.id }}})</h2>
<img [src]="pokemon.sprites.front_default" alt="{{pokemon.name}}" />
<p>Tipos: <span *ngFor="let t of types">{{ t }} </span></p>
<ul>
  <li *ngFor="let s of mainStats">{{ s.name }}: {{ s.value }}</li>
</ul>
</div>

```

Explicación:

El componente recibe un objeto Pokemon mediante @Input() y muestra sus propiedades. Usa pipes (titlecase) para mejorar la presentación.

Paso 8. Integrar los componentes en AppComponent

Ruta: src/app/app.component.html

```

<div class="container">
  <h1>Buscador de Pokémon</h1>
  <app-search></app-search>
</div>

```

Explicación:

Se coloca el componente principal <app-search> en la plantilla raíz para que gestione todo el flujo de búsqueda.

Paso 9. Ejecutar y probar la aplicación

ng serve

Abrir el navegador en <http://localhost:4200> e ingresar un nombre o número (por ejemplo: *pikachu* o *25*).

Si la conexión con la API es exitosa, se mostrará la información del Pokémon consultado.

6. Bibliografía

- PokéAPI, RESTful Pokémon API, [Online]. Disponible en: <https://pokeapi.co>
- Angular Documentation, Request data from a server, Angular.io, 2025. Disponible en: <https://angular.io/guide/http>

7. Anexos

A. Fragmento del archivo package.json:

```

{
  "name": "pokemon-search",
  "version": "0.0.1",
  "scripts": {
    "start": "ng serve",
    "build": "ng build"
  },
  "dependencies": {
    "@angular/common": "^17.0.0",
    "@angular/core": "^17.0.0",
    "rxjs": "^7.0.0"
  }
}

```

```
}  
}
```

Anexo B: Estructura del Proyecto

```
src/  
├── app/  
│   ├── services/  
│   │   └── pokemon.service.ts  
│   ├── components/  
│   │   ├── search/  
│   │   └── pokemon-card/  
│   ├── app.component.ts  
│   ├── app.component.html  
│   └── app.module.ts
```

Anexo C: Ejemplo de Ejecución

Vista del navegador al buscar *pikachu* mostrando nombre, imagen, tipos y estadísticas básicas.

Anexo D: Verificación de la API con Postman

Antes de integrar la API en Angular, es recomendable **comprobar su conectividad** y estructura de respuesta usando **Postman**.

Esto garantiza que el endpoint funcione correctamente y que los datos retornados sean válidos para su manipulación en el servicio `pokemon.service.ts`.

Pasos:

1. Abrir **Postman** e iniciar una nueva petición.
2. Configurar el método como **GET**.
3. Introducir la siguiente URL:
4. `https://pokeapi.co/api/v2/pokemon/pikachu`
5. Hacer clic en **Send**.
6. Revisar la respuesta JSON en la pestaña **Body**:

Respuesta esperada:

```
{  
  "id": 25,  
  "name": "pikachu",  
  "sprites": { "front_default": "https://raw.githubusercontent.com/.../pikachu.png" },  
  "types": [ { "type": { "name": "electric" } } ],  
  "stats": [ ... ]  
}
```

Validación:

- Código de estado: 200 OK
- Tiempo de respuesta: < 500 ms
- Cuerpo de respuesta: JSON estructurado con los campos `id`, `name`, `sprites`, `types`, `stats`

☒ Si se obtiene esta respuesta, la API está operativa y lista para integrarse con Angular.

⚠ En caso de recibir 404 Not Found o 429 Too Many Requests, revisar el nombre del Pokémon o esperar algunos segundos antes de reenviar la petición.